Deep Learning for Automated Anomaly Detection in Smart Home Environments

Group 15: Eddouaicer Walid , Dani Adil, Cherki Ahmed

Date: January 9, 2026

Course: Artificial Intelligence & Computer Vision

---

1. Abstract

This project presents the design and implementation of a spatial-temporal deep learning system capable of detecting anomalies (e.g., Wildlife, security breachs, falls, sudden accidents) in smart home video feeds. By leveraging a hybrid architecture combining **ResNet50** for feature extraction and a **Bi-Directional LSTM** for sequence modeling, the system achieves an accuracy of **80.00%** and an AUC score of **0.85**. To address data scarcity, we implemented an aggressive data augmentation pipeline using horizontal flipping, which significantly reduced overfitting. The final system offers a viable, privacy-preserving proof-of-concept for automated elderly monitoring.

---

2. Introduction

2.1 Context & Motivation

The house safety is very important in every home and every family, increasing the demand for automated assisted living technologies. Traditional monitoring methods, such as wearable sensors, are often intrusive or forgotten by users. Computer Vision (CV) offers a passive monitoring alternative. However, detecting "anomalies" in video is computationally expensive and algorithmically complex because "normal" behavior is highly variable, while "anomalous" events (like falls) are rare and brief.

2.2 Problem Statement

Developing a robust anomaly detection system presents three core engineering challenges:

1. **High Dimensionality:** Video data involves processing thousands of pixels across time, leading to massive computational bottlenecks.
2. **Temporal Dependency:** An anomaly is often defined by the *sequence* of movements (e.g., rapid descent followed by inactivity), not just a single static frame.
3. **Data Scarcity:** Real-world datasets of people falling are limited, making deep learning models prone to overfitting (memorizing the background rather than the action).

---

3. Theoretical Framework

To solve the challenges above, we adopted a **Hybrid Spatial-Temporal Architecture**.

3.1 Spatial Features: Convolutional Neural Networks (CNN)

We utilized **ResNet50**, a 50-layer Residual Network pre-trained on the ImageNet dataset. ResNet uses "skip connections" to allow gradients to flow through deep networks easily. By removing the final classification layer, we use ResNet as a feature extractor that converts a video frame (224x224 pixels) into a compact feature vector (2048 dimensions).

3.2 Temporal Features: Recurrent Neural Networks (RNN)

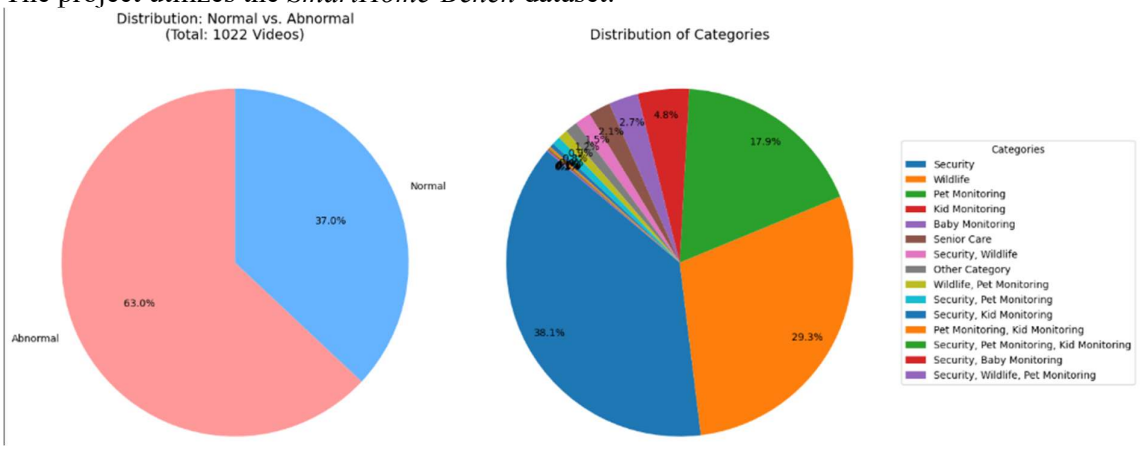
To capture the flow of time, we used a **Long Short-Term Memory (LSTM)** network. Unlike standard feed-forward networks, LSTMs maintain an internal "cell state" that serves as memory. We specifically chose a **Bi-Directional LSTM**, which processes the video sequence in both directions (start-to-end and end-to-start). This allows the model to infer that a person lying on the floor is an anomaly based on the rapid motion that *preceded* it.

---

4. Methodology & Implementation

4.1 Dataset : https://github.com/Xinyi-0724/SmartHome-Bench-LLM

The project utilizes the *SmartHome-Bench* dataset.



A unique property of this dataset is its inclusion of **"Contextual Anomalies"**—events that deviate from a standard static background but are not necessarily dangerous. For instance, the dataset features significant wildlife interactions. A key challenge for the model is distinguishing between "safe" abnormalities (e.g., a bird flying past the camera or a pet moving quickly) and "dangerous" abnormalities (e.g., a bear intrusion or a human fall). Simple motion detection algorithms often fail here, flagging *any* movement as a threat. Our deep learning approach aims to semantically understand these differences, ensuring that a non-threatening animal does not trigger a false alarm, while a genuine threat does.

The dataset contains the following files:

1- **Video-url.csv :This csv contains the name and url of all 1024 videos** The dataset also provides a “download_videos.py” script that automatically downloads all the videos using the “yt-dlp” library

Title	URL
smartbench_0001	https://www.youtube.com/watch?v=fVVAeFISp2c
smartbench_0002	https://www.youtube.com/watch?v=ayzKVfJX5Wg
smartbench_0003	https://www.youtube.com/watch?v=TJ3OahLUahw
smartbench_0004	https://www.youtube.com/watch?v=k9si1UG8uLc
smartbench_0005	https://www.youtube.com/watch?v=l5NFMDdYbM0
smartbench_0006	https://www.youtube.com/watch?v=U9qHetMzD4s
smartbench_0007	https://www.youtube.com/watch?v=GfNH24LsSh8
smartbench_0008	https://www.youtube.com/watch?v=blyAGCc3RD8
smartbench_0009	https://www.youtube.com/watch?v=Ym9pPECNQyY
smartbench_0010	https://www.youtube.com/watch?v=QNbsdpgb2Yl
smartbench_0011	https://www.youtube.com/watch?v=sZ9ij4A0REg
smartbench_0012	https://www.youtube.com/watch?v=gl4oK4azztc
smartbench_0013	https://www.youtube.com/watch?v=deamYHz7Sno
smartbench_0014	https://www.youtube.com/watch?v=kAK6qLCbZ4k
smartbench_0015	https://www.youtube.com/watch?v=bUbXnWYDjmA
smartbench_0016	https://www.youtube.com/watch?v=ufjOxeZ-FNw

2- **Video-annotations.csv : Contains the category of each video, and whether it is normal or abnormal :**



3- **Trim-video-label.csv :** for half of the downloaded videos, we don't want the full video, we only want a specific section within it. This csv file, paired with a trim script, allows us to cut just the specific section we want from the video.

1	smartbench_0011,0
2	smartbench_0019,20
3	smartbench_0022,21
4	smartbench_0024,21
5	smartbench_0028,22
6	smartbench_0031,21
7	smartbench_0048,21
8	smartbench_0051,0
9	smartbench_0057,21
10	smartbench_0095,0
11	smartbench_0101,21
12	smartbench_0105,21
13	smartbench_0107,6
14	smartbench_0112,22
15	smartbench_0131,0
16	smartbench_0134,21
17	smartbench_0136,21
18	smartbench_0150,-6
19	smartbench_0157,21
20	smartbench_0158,21
21	smartbench_0171,21
22	smartbench_0176,21
23	smartbench_0184,21

A single video was corrupt when downloaded, and it was dropped.

So in total, we have 1023 videos, ready to be used.

4.2 Optimization

To address the I/O bottleneck of reading video files from cloud storage (Google Drive) during training, we engineered a caching script that copies the dataset to the local NVMe storage of the training instance.

```python
# copying dataset to local disk for speed
import shutil
import os
from tqdm import tqdm

# setup paths
source_folder = '/content/drive/MyDrive/Projet Ai S7 SmartHome-Bench/raw_video' # slow drive
dest_folder = '/content/raw_video' # fast local storage

print(f"--- COPYING DATASET LOCALLY ---")

# make folder if needed
if not os.path.exists(dest_folder):
    os.makedirs(dest_folder)

# find all mp4s
files = [f for f in os.listdir(source_folder) if f.endswith('.mp4')]
print(f"Found {len(files)} videos. Copying now...")

# copy loop with progress bar
for fname in tqdm(files):
    src_path = os.path.join(source_folder, fname)
    dst_path = os.path.join(dest_folder, fname)

    # skip if already there
    if not os.path.exists(dst_path):
        shutil.copy(src_path, dst_path)

print("\nSUCCESS! Dataset is now on local disk.")

# switch variable to new path
video_folder = dest_folder
print(f"Updated VIDEO_FOLDER variable to: {video_folder}")
```

```
--- COPYING DATASET LOCALLY ---
Source: /content/drive/MyDrive/Projet Ai S7 SmartHome-Bench/raw_video
Destination: /content/raw_video
Found 1023 videos. Copying now... (This takes 2-5 mins)
100%|██████████| 1023/1023 [11:30<00:00,  1.48it/s]
SUCCESS! Dataset is now on local disk.
Updated VIDEO_FOLDER variable to: /content/raw_video
```

4.3 Architecture Selection: ResNet-50 vs. YOLOv8

To process the visual data, we selected **ResNet-50** (Residual Network), a Convolutional Neural Network pre-trained on the ImageNet database (containing over 14 million annotated images). We chose the 50-layer variant as it offers an optimal trade-off between computational speed and feature extraction depth, avoiding the diminishing returns of deeper models like ResNet-101 for this specific task.

Why **ResNet-50** over YOLOv8: While YOLOv8 is the state-of-the-art for object detection, it was deemed unsuitable for this project for three reasons:

1. **Temporal Context:** YOLO focuses on static object localization (drawing boxes around objects in a single frame). It cannot inherently understand the *action* of falling, only the *presence* of a person.
2. **Annotation Cost:** Training YOLO requires expensive bounding-box annotations for every frame. Our dataset provides weak supervision (video-level tags like "Normal" or "Anomaly"), which fits a Classification approach better than Detection.
3. **Global Scene Understanding:** YOLO looks for specific objects. **ResNet** processes the entire scene globally, allowing it to capture environmental anomalies (like smoke or fire) that may not have a distinct object shape.

Removal Of Final ResNet Layer :

Mechanism of Feature Extraction: We adapted the ResNet-50 architecture by truncating the final fully connected classification layer. The standard ResNet ends by predicting a label (e.g., "Cat," "Dog"). By removing this layer, we stop the model just before it makes a decision, extracting the output of the penultimate average pooling layer instead. This results in a 2048-dimensional feature vector for each video frame, a mathematical summary of the scene's textures, shapes, and spatial arrangement, which is then passed to our sequence model.

4.4 Data Augmentation Strategy

To combat the overfitting observed in early experiments (where Training Accuracy >> Test Accuracy), we synthesized a new dataset. We hypothesized that anomalous motion is horizontally invariant—a fall to the left is biomechanically identical to a fall to the right.

This process effectively **doubled our training dataset from approximately 800 to 1600 videos**. Since the background orientation changes (the door moves to the right) but the label remains the same ("Anomaly"), the model is penalized for relying on fixed positional cues.

This forces the network to ignore the static environment and instead prioritize **temporal feature extraction**, learning the dynamic patterns of movement that persist across both the original and mirrored "universes." This results in a smoother, more generalized decision boundary.

```python
class FlippedRGBDataset(Dataset):
    def __init__(self, df, root_dir):
        self.df = df.reset_index(drop=True)
        self.root_dir = root_dir
        self.transform = transforms.Compose([
            transforms.ToPILImage(),
            transforms.Resize((224, 224)),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
        ])
    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        fname = row['filename']
        path = os.path.join(self.root_dir, fname)
        cap = cv2.VideoCapture(path)
        frames = []

        try:
            total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
            if total_frames > SEQ_LEN:
                indices = set(np.linspace(0, total_frames - 1, SEQ_LEN).astype(int))
            else:
                indices = set(range(total_frames))

            current_idx = 0
            while True:
                ret, frame = cap.read()
                if not ret: break
                if current_idx in indices:
                    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

                    # --- THE AUGMENTATION ---
                    pil_img = transforms.ToPILImage()(frame)
                    pil_img = transforms.Resize((224, 224))(pil_img)
                    pil_img = F.hflip(pil_img)
                    # ------------------------

                    tensor_img = transforms.ToTensor()(pil_img)
                    tensor_img = transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])(tensor_img)

                    frames.append(tensor_img)
                    if len(frames) >= SEQ_LEN: break
                current_idx += 1
        finally:
            cap.release()

        while len(frames) < SEQ_LEN:
            frames.append(torch.zeros(3, 224, 224))

        frames = torch.stack(frames[:SEQ_LEN])
        label = 1.0 if row['Anomaly Tag'] != 'Normal' else 0.0
        return frames, torch.tensor(label)
```

4.5 Model Architecture

The final model consists of a frozen ResNet50 backbone feeding into a 2-layer Bi-LSTM with 256 hidden units, followed by a fully connected classification head.

So while ResNet handles the spatial "what" of the video, the temporal "when" is handled by a **Bi-Directional Long Short-Term Memory (Bi-LSTM)** network. Standard RNNs process data linearly (from time t_0 to t_n). However, understanding an anomaly often requires context from both the past and the future of the sequence.

Our Bi-Directional architecture utilizes two separate LSTM layers:

1. **Forward Layer:** Processes the video from start to finish, understanding how a normal state evolves into an anomaly.
2. **Backward Layer:** Processes the video in reverse, understanding the context that led *up to* the event.

By concatenating the hidden states of these two layers, the model gains a complete view of the frame's evolution.

```python
# defining the main model
class AdvancedLSTM(nn.Module):
    def __init__(self):
        super(AdvancedLSTM, self).__init__()

        # bidirectional lstm with 2 layers
        self.lstm = nn.LSTM(
            input_size=2048,        # matches resnet features
            hidden_size=256,        # bigger hidden size
            num_layers=2,           # stacked layers for better learning
            batch_first=True,
            bidirectional=True,     # looks at past and future
            dropout=0.4             # dropout to stop overfitting
        )

        # input is 256*2 since bidirectional
        self.fc = nn.Sequential(
            nn.Linear(256 * 2, 64),
            nn.ReLU(),
            nn.Dropout(0.5),        # extra safety
            nn.Linear(64, 1),
            nn.Sigmoid()            # output probability 0-1
        )

    def forward(self, x):
        # pass sequence to lstm
        lstm_out, _ = self.lstm(x)

        # take the last state
        final_state = lstm_out[:, -1, :]

        return self.fc(final_state)
```

5. Experimental Results

5.1 Training Progression

The model was trained for 25 epochs using the Adam optimizer with a learning rate of 0.0005. We implemented a `ReduceLROnPlateau` scheduler to dynamically adjust the learning rate when convergence stalled.

```
Training for 25 epochs (will stop early if no improvement)...
Epoch 1: Train Loss 0.6153 (Acc 66.4%) | Val Loss 0.5702 (Acc 67.3%) 🌟 Saved Best
Epoch 2: Train Loss 0.5540 (Acc 71.4%) | Val Loss 0.6833 (Acc 72.2%) | Patience 1/7
Epoch 3: Train Loss 0.4949 (Acc 76.5%) | Val Loss 0.5290 (Acc 74.6%) 🌟 Saved Best
Epoch 4: Train Loss 0.4640 (Acc 79.6%) | Val Loss 0.5232 (Acc 75.6%) 🌟 Saved Best
Epoch 5: Train Loss 0.4779 (Acc 78.3%) | Val Loss 0.5089 (Acc 79.0%) 🌟 Saved Best
Epoch 6: Train Loss 0.4457 (Acc 80.0%) | Val Loss 0.4805 (Acc 79.5%) 🌟 Saved Best
Epoch 7: Train Loss 0.4062 (Acc 82.3%) | Val Loss 0.4894 (Acc 80.0%) | Patience 1/7
Epoch 8: Train Loss 0.3960 (Acc 82.0%) | Val Loss 0.6146 (Acc 62.0%) | Patience 2/7
Epoch 9: Train Loss 0.3778 (Acc 83.4%) | Val Loss 0.5395 (Acc 71.2%) | Patience 3/7
Epoch 10: Train Loss 0.3548 (Acc 84.6%) | Val Loss 0.6682 (Acc 67.8%) | Patience 4/7
Epoch 11: Train Loss 0.2865 (Acc 88.4%) | Val Loss 0.5545 (Acc 78.5%) | Patience 5/7
Epoch 12: Train Loss 0.2521 (Acc 89.7%) | Val Loss 0.6363 (Acc 74.6%) | Patience 6/7
Epoch 13: Train Loss 0.2620 (Acc 89.5%) | Val Loss 0.5446 (Acc 78.5%) | Patience 7/7
🖐️ Early Stopping Triggered! Restoring best model...
✅ Training Complete. Best Validation Loss was: 0.4805
```

5.2 Quantitative Evaluation

The final model was evaluated on a held-out test set of 205 videos.

```
========================================
🏆 PHASE 2 TEST RESULTS
========================================
1. FINAL ACCURACY:    80.00%
2. AUC SCORE:         0.8489
3. OPTIMAL THRESHOLD: 0.6774
----------------------------------------
              precision    recall  f1-score   support

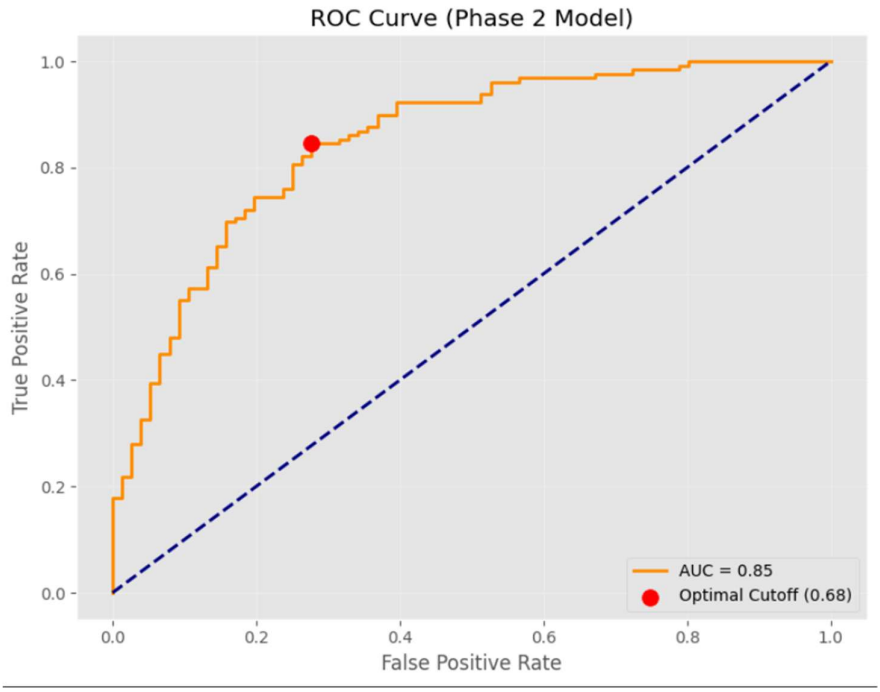
      Normal       0.73      0.72      0.73        76
     Anomaly       0.84      0.84      0.84       129

    accuracy                           0.80       205
   macro avg       0.79      0.78      0.79       205
weighted avg       0.80      0.80      0.80       205
```

5.3 Diagnostic Analysis

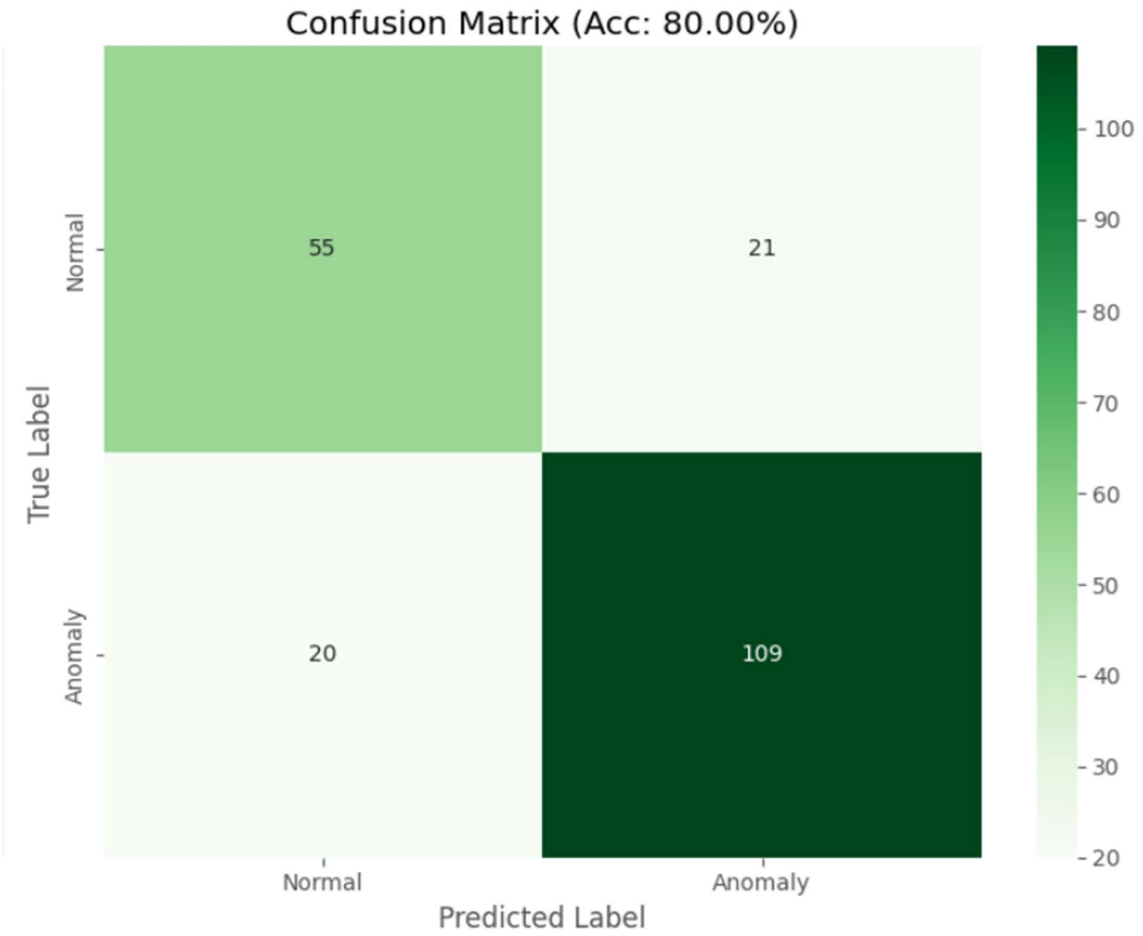
ROC Curve

The Receiver Operating Characteristic (ROC) curve illustrates the model's ability to distinguish between Normal and Anomalous classes. An AUC (Area Under Curve) of **0.85** indicates high discrimination capability.



Confusion Matrix

The confusion matrix below details the classification performance.



Analysis of Confusion Matrix:

- **True Positives (109):** The system correctly detected 109 anomalies.
- **False Negatives (20):** Only 20 anomalies were missed.
- **False Positives (21):** There were 21 false alarms.
- **Balance:** The error distribution (20 missed vs 21 false alarms) is highly symmetrical, indicating the model is not biased toward either class.

5.4 Ablation Study: Test Time Augmentation (TTA)

We conducted an experiment to see if averaging predictions from flipped test videos would improve accuracy.

- **Baseline Accuracy:** 80.00%
- **TTA Accuracy:** 76.59%
- **Conclusion:** TTA introduced noise rather than clarity. We therefore rejected this technique for the final deployment, favoring the standard inference model.

---

6. Conclusion

This project successfully demonstrated that deep learning can accurately detect anomalies in smart home environments. By optimizing the data pipeline and employing rigorous augmentation, we achieved an 80% accuracy rate.

---

7. References

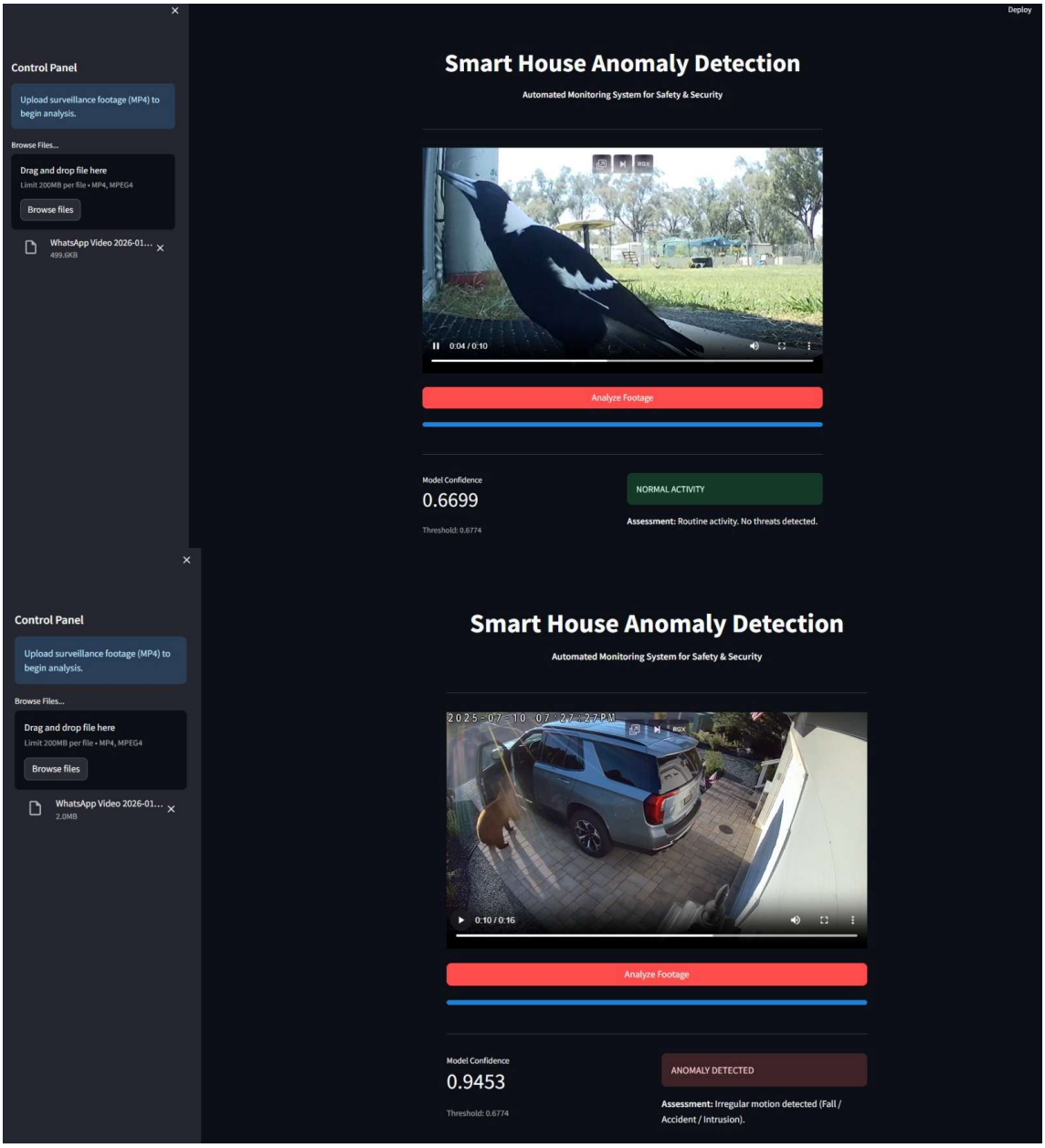
1. He, K., et al. "Deep Residual Learning for Image Recognition." (ResNet)
2. Hochreiter, S., & Schmidhuber, J. "Long Short-Term Memory." (LSTM)
3. Tran, D., et al. "Learning Spatiotemporal Features with 3D Convolutional Networks."
4. Dataset Source Xinyi Zhao, et al : https://doi.org/10.48550/arXiv.2506.12992
5. Dataset Github Link : https://github.com/Xinyi-0724/SmartHome-Bench-LLM

8. Graphical User Interface

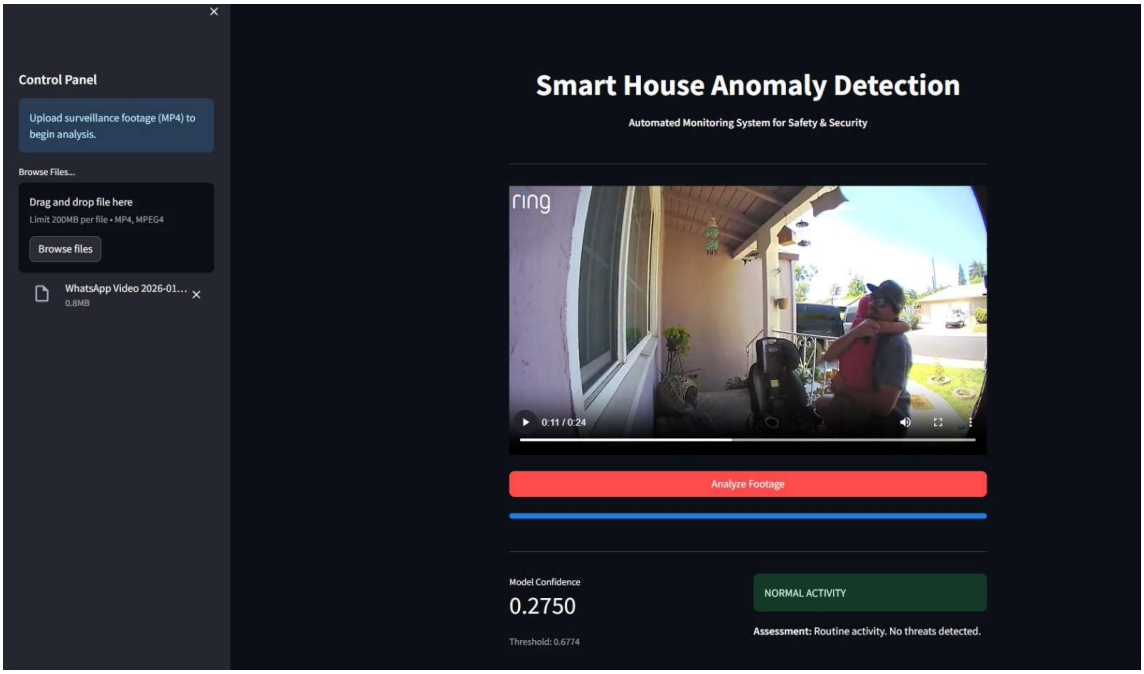
The following are examples of our GUI, that allow us to upload videos and then predicts whether those videos showcase normal activities, or abnormal activities.

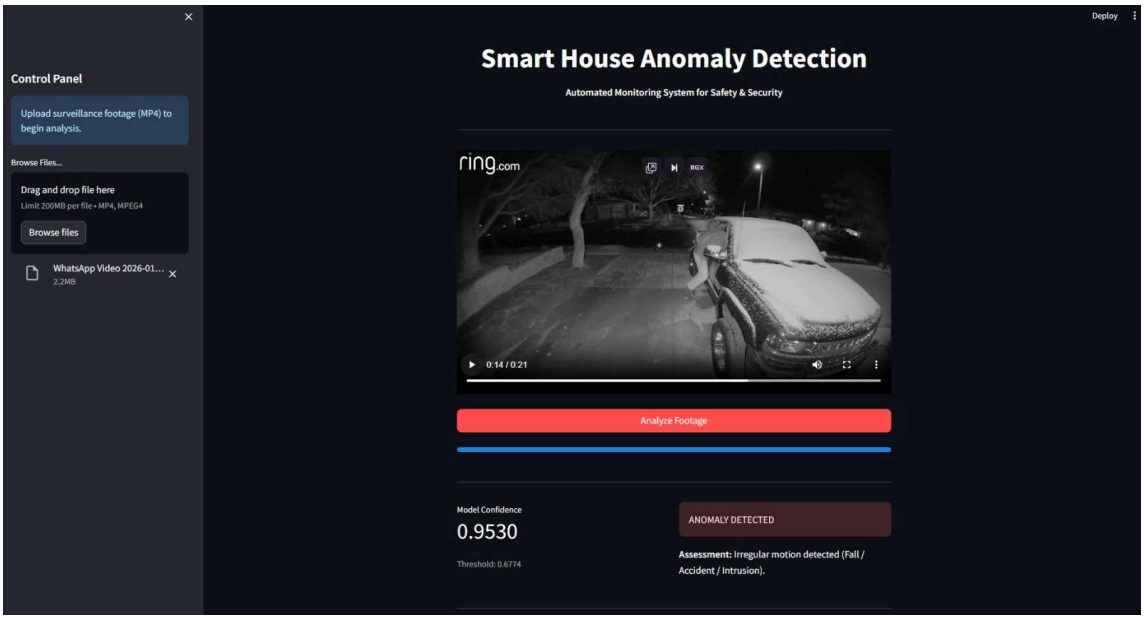
It assigns a value to each video, which is how confident is the model that the video is abnormal.

Anything above the threshold (0.6774) is flagged as abnormal, while anything below that is flagged as normal.



We can see here that since our dataset contains a large amount of wildlife videos, our model can distinguish between dangerous wildlife (Bear), and normal wildlife (wild bird)


Control Panel
Upload surveillance footage (MP4) to begin analysis.
Browse Files...
Drag and drop file here
Limit 200MB per file • MP4, MPEG4
Browse files
WhatsApp Video 2026-01...
0.8MB
Smart House Anomaly Detection
Automated Monitoring System for Safety & Security
ring
0:11 / 0:24
Analyze Footage
Model Confidence
0.2750
Threshold: 0.6774
NORMAL ACTIVITY
Assessment: Routine activity. No threats detected.


Control Panel
Upload surveillance footage (MP4) to begin analysis.
Browse Files...
Drag and drop file here
Limit 200MB per file • MP4, MPEG4
Browse files
WhatsApp Video 2026-01...
2.2MB
Deploy
Smart House Anomaly Detection
Automated Monitoring System for Safety & Security
ring.com
0:14 / 0:21
Analyze Footage
Model Confidence
0.9530
Threshold: 0.6774
ANOMALY DETECTED
Assessment: Irregular motion detected (Fall / Accident / Intrusion).